



عملي

7

18

2160 sp

كلية الهندسة المعلوماتية

السنة الثانية

تعليمات المقارنة وتحميل وتخزين البايتات د. عبير ميا



RB Informatics; 17/06/2025

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

تعليمات المقارنة

1. slt: (Set if Less Than)

Syntax:

slt rd, rs, rt

وهي تعليمة حقيقية من نمط R-Type، تقوم بجعل سجل الوجهة rd = 1 إذا تحقق: $rs < rt$ ، وإلا فتجعل rd = 0. rd ←

2. slti: (Set if Less Than Immediate)

Syntax:

slti rt, rs, immediate

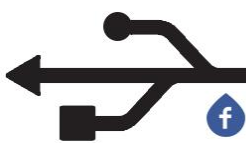
وهي أيضا تعليمة حقيقية من نوع I-Type، تقوم بعمل مشابه لعمل slt حيث تجعل قيمة السجل rt = 1 في حال تحقق $rs < immediate$ وإلا فتجعل rt = 0.

3. sltu: (Set if Less Than Unsigned)

Syntax:

sltu rd, rs, rt

وهي أيضا تعليمة حقيقية من نوع R-Type، تقوم بنفس عمل slt تماما لكنها تنظر إلى محتوى السجلات على أنه بلا إشارة (unsigned).



■ ملاحظة:

إن قراءة قيمة من سجل تختلف حسب التمثيل التي تعتمد التعليمات.
 بفرض أن الأعداد تخزن على 4bit (كمثال)، بالتالي يمكننا تمثيل $2^4 = 16$ عدد على هذه البتات. وهنا نميز حالتين:
 تمثيل أعداد مع إشارة (signed) أو تمثيل أعداد بدون إشارة (unsigned):

Unsigned

هنا يتم اعتبار جميع الأعداد موجبة
 ← الـ 16 عدد الممكن تمثيلهم على
 4bit سيكونون في القسم الموجب جميعاً.
 [0, 1, 2, ..., 6, 7, ..., 14, 15]
 الأعداد التي يمكن تمثيلها بـ 4 bit
 (unsigned)

Signed

هنا يتم أخذ الأعداد الموجبة والسالبة
 بعين الاعتبار ← الـ 16 عدد الممكن
 تمثيلهم سينقسمون بين السالب
 والموجب
 كالتالي:
 [-8, -7, -6, ..., 0, 1, 2, ..., 6, 7]
 الأعداد التي يمكن تمثيلها بـ 4 bit

مثال:

ليكن لدينا العدد $(1011)_2$
 إذا كانت التعليمات تقرأ الأعداد على أنها signed
 ستقرأ $(1011)_2$ على أنها العدد السالب $(-5)_{10}$.
 إذا كانت التعليمات تقرأ الأعداد على أنها unsigned ستقرأ العدد
 $(1011)_2$ على أنها unsigned فستقرأ العدد $(1011)_2$ على أنه موجب
 وقيمة $(11)_{10}$
 نلاحظ أن العدد نفسه تختلف قراءته حسب التمثيل الذي
 تعتمد التعليمات.



Binary	Signed	Unsigned
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	-8	8
1001	-7	9
1010	-6	10
1011	-5	11
1100	-4	12
1101	-3	13
1110	-2	14
1111	-1	15

تذكّرة بالتحويل:

1. إذا كان تمثيل العدد unsigned فاننا نحول من Binary إلى Decimal بشكل طبيعي.
في المثال السابق:

$$(1011)_2 = 1 \times 2^3 + 1 \times 2 + 1 \times 2^0 = 8 + 2 + 1 = (11)_{10}$$

2. إذا كان تمثيل العدد signed:

ننظر إلى آخر بت (بت الإشارة) إذا كان 1 فالعدد سالب إذا كان 0 فالعدد موجب.

ملاحظة: هذا البت يدل على إشارة العدد لكنه ليس محجزوًا لها

إذا كان العدد موجب $\Leftarrow$ نحول مباشر إلى Decimal

إذا كان العدد سالب $\Leftarrow$ نقوم بتحويل الأصفار إلى واحد و الواحدات إلى أصفار ثم إضافة 1 (المتمم الثنائي)

$$(1011)_2 \Rightarrow (0100)_2 \Rightarrow (0101)_2$$

نحول الناتج إلى Decimal:

$$(0101)_2 = 1 \times 2^2 + 1 \times 2^0 = 4 + 1 = 5$$

وبما ان الإشارة سالبة:

$$(1011)_2 = (-5)_{10}$$

تمرين:

بفرض ان:

$$\$t_1 = 1010\ 1010\ 1010\ 1010\ 1111\ 0000\ 1111\ 0000$$

ما هو محتوى سجلات الوجهة بعد تنفيذ التعليمات التالية:

▪ `slt $t4, $t1, $0`

الحل:

slt تستخدم تمثيل الأعداد مع إشارة signed $\Leftarrow$ ستقرأ محتوى \$t1 على أنه عدد سالب (sign bit = 1) ومنه $\$t1 < 0$ (الشرط محقق) $\Leftarrow \$t4 = 1$

▪ `slti $t5, $t1, 0xF`

الحل:

slti تنظر إلى الأعداد على انها اشارة (signed) $\Leftarrow$ يكون تمثيل 0xF على 32bit:

$$(0xF)_{16} = (0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1111)_2$$

فلاحظ أن 0xF عدد موجب، بينما \$t1 تحوي على عدد سالب.

ومنه: $\$t1 < 0xF$ (الشرط تحقق) $\Leftarrow \$t5 = 1$

▪ `sltu $t6, $t1, $0`

sltu تتعامل مع الأعداد على أنها unsigned $\Leftarrow$ ستقرأ محتوى \$t1 على انه موجب،

ومنه: $\$t1 > 0$ (الشرط غير محقق) $\Leftarrow \$t6 = 0$

تعليمات المقارنة مع التفرع: (تعليمات زائفة)

في MIPS يوجد فقط تعليمة حقيقية لعملية المقارنة أصغر من ($<$) وهي `slt` بأشكالها المختلفة. أما عمليات المقارنة الأخرى ($>$, $>=$, $<=$) فهي عبارة عن `slt` مجتمعة مع التعليمات الشرطية حتى تعطي جميع الحالات

1. `blt`: (Branch if Less Than)

Syntax:

`blt rt, rs, Label` # if ($rt < rs$) go to "label"

Example:

إذا كان $\$a0 < \$a1$ فيتم الانتقال إلى التعليمة `Label` وإلا يتم إكمال التعليمات بالتسلسل. التعليمات الحقيقية التي تكافئها: (`slt + bne` + سجلات) (بدون عكس السجلات)

نلاحظ أن السجلات لم تعكس هنا

```
blt $a0, $a1, label
slt $at, $a0, $a1      # if ($a0 < $a1) {$at = 1} else {$at = 0}
bne $at, $0, label     # if ($at != 0) {go to "label"} else { continue sequentially }
```

2. `ble`: (Branch if Less Than or Equal to)

Syntax:

`ble rt, rs, label` # if ($rt \leq rs$) go to "label"

Example:

إذا كان $\$a0 \leq \$a1$ فيتم الانتقال إلى التعليمة `label` وإلا فيتم إكمال التعليمات بالتسلسل. التعليمات الحقيقية التي تكافئها: (`slt + beq` + سجلات) (مع عكس السجلات)

نلاحظ عكس السجلات هنا

```
ble $a0, $a1, label
slt $at, $a1, $a0      # if ($a1 < $a0) {$at = 1} else {$at = 0}
beq $at, $0, label     # if ($at == 0) {go to "label"} else { continue sequentially }
```

3. `bgt`: (Branch if Greater Than)

Syntax:

`bgt rt, rs, Label` # if ($rt > rs$) go to "label"

Example:

إذا كان $\$a0 > \$a1$ فيتم الانتقال إلى التعليمة `Label` وإلا يتم إكمال التعليمات بالتسلسل. التعليمات الحقيقية التي تكافئها: (`slt + bne` + سجلات) (مع عكس السجلات)

نلاحظ عكس السجلات هنا

```
bgt $a0, $a1, label
slt $at, $a1, $a0      # if ($a1 < $a0) {$at = 1} else {$at = 0}
bne $at, $0, label     # if ($at != 0) {go to "label"} else { continue sequentially }
```

4. bge: (Branch if Greater Than or Equal to)

Syntax:

bge rt, rs, label *#if (rt >= rs) go to label*

Example:

bge \$a0, \$a1, label

إذا كان $a0 \geq a1$ فيتم الانتقال إلى التعليمة label وإلا فيتم إكمال التعليمات بالتسلسل. نلاحظ أن السجلات

التعليمات الحقيقية التي تكافئها: (بدون عكس السجلات + slt + beq)

slt \$at, \$a0, \$a1

if (\$a0 < \$a1) {\$at = 1} else {\$at = 0}

beq \$at, \$0, label

if (\$at == 0) {go to "label"} else {continue sequentially}

تعليمات تحميل وتخزين بايت

تشبه التعليمات lw و sw لكن بدلاً من التعامل مع كلمة 4 byte فهنا نتعامل مع 1 byte. بما أن السجل مؤلف من 4 byte، فتم تخصيص البايت الأدنى lowest byte للتعامل مع هذه التعليمات.

1. lb: (load byte)

Syntax:

lb \$rt, offset(\$rs)

يتم تحميل 1 byte من العنوان M[offset + \$rs] ونخزينه في الـ lowest byte في السجل \$rt

2. sb: (store byte)

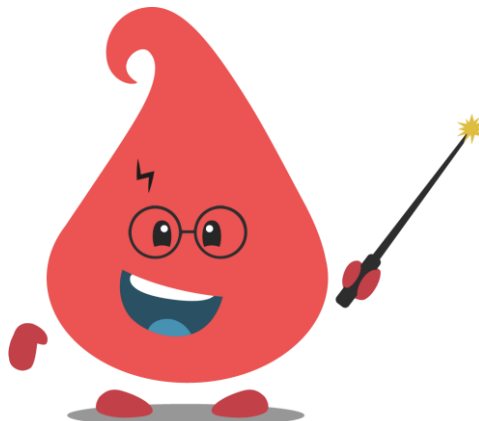
Syntax:

sb \$rt, offset(\$rs)

يتم تخزين محتوى البايت الأدنى من السجل \$rt في العنوان M[offset + \$rs]

■ ملاحظة اربسية:

الـ offset في lb و sb يمكن ان يكون أي عدد لأننا نتعامل مع bytes. على عكس الـ offset في lw و sw والذي يجب ان يكون من مضاعفات الأربعة لأننا نتعامل مع words.



```
.data
bt: .byte 'A', '0', 'a', '9' # storing these values in the memory, each one is 1 byte
sp: .space 4 # reserve 4 uninitialized bytes

.text
la $s2, bt # load bt address into $s2 --> $s2 = 0x1001 0000
la $s1, sp # load sp address into $s1 --> $s1 = 0x1001 0004
lb $t0, 3($s2) # $t0 <-- M[3 + $s2] => $t0 <-- M[0x1001 0003] => $t0 = '9'
sb $t0, 0($s1) # $t0 --> M[0 + $s1] => '9' --> M[0x1001 0004]
lb $t0, 2($s2) # $t0 <-- M[2 + $s2] => $t0 <-- M[0x1001 0002] => $t0 = 'a'
sb $t0, 3($s1) # $t0 --> M[3 + $s1] => 'a' --> M[0x1001 0007]
```

bt فيها 4 عناصر كل منها 1 byte، يبدأ تخزينها من 0x1001 0000.
 sp سنبدأ تخزين هذه البايتات مباشرة بعد آخر عنصر في bt أي أن التخزين سيبدأ من 0x1001 0004.

الذاكرة قبل تنفيذ البرنامج:

Data Segment		
Address	Value (+0)	Value (+4)
0x10010000	9 a 0 A	\0 \0 \0 \0

الذاكرة بعد تنفيذ البرنامج

Data Segment		
Address	Value (+0)	Value (+4)
0x10010000	9 a 0 A	a \0 \0 9

نلاحظ أن تخزين البايتات يبدأ من البت الأدنى.

مثال أربسيزي:

Code:

```
.data
s: .ascii "hello:)"
```

Output:

Data Segment		
Address	Value (+0)	Value (+4)
0x10010000	l l e h	\0) : o

ملاحظة: عند تحميل 1 byte في الذاكرة إلى سجل، فيتم تخزينه في البايت الأدنى للسجل (من البت 0 ← 7)، أما باقي البتات فيتم ملؤها كالتالي:

1- إذا كنا نستخدم التعليمة lb (sign extended): فيتم ملء بقية البتات (من البت 8 ← 31) حسب قيمة البت 7 (وهو بت الإشارة sign bit).

2- إذا كنا نستخدم التعليمة lbu (zero extended): فيتم ملء بقية البتات بالقيمة 0.

مثال اربسيزي:

■ بفرض لدينا القيمة $(1111\ 1000)_2$ مخزنة في عنوان معين في الذاكرة ونريد تحميل هذا البايت إلى السجل \$s0:

إذا استخدمنا **lb** فسيكون محتوى السجل:

\$s0 = 1111 1111 1111 1111 1111 1111 1111 1000

قيمتهم مثل البت 7 7th bit Lowest byte

إذا استخدمنا **lbu** فسيكون محتوى السجل:

\$s0 = 0000 0000 0000 0000 0000 0000 0000 1111 1000

قيمتهم دائماً 0

■ بفرض لدينا القيمة $(0000\ 0100)_2$ مخزنة في عنوان معين في الذاكرة ونريد تحميل هذا البايت إلى السجل \$s0:

إذا استخدمنا **lb** فسيكون محتوى السجل:

\$s0 = 0000 0000 0000 0000 0000 0000 0000 0000 0100

قيمتهم مثل البت 7 7th bit

إذا استخدمنا **lbu** فسيكون محتوى السجل:

\$s0 = 0000 0000 0000 0000 0000 0000 0000 0000 0100

قيمتهم دائماً 0

■ تمرين 1:

اكتب برنامج بلغة التجميع MIPS يقوم بتحويل سلسلة أحرف كبيرة مخزنة في الذاكرة بدءاً من العنوان القاعدي \$s1 إلى أحرف صغيرة وحفظها بدءاً من موقع الذاكرة ذي العنوان القاعدي \$s2. علماً أن سلسلة المحارف تنتهي بـ null.

ملاحظات

1- الفرق بين الأحرف الكبيرة والصغيرة هي ascii هو:

Small letter = Capital letter + 32

تمثيل null في ascii هو 0. (أما تمثيل الـ 0 هو 48)

2- بما أن سلسلة المحارف تنتهي بـ null $\Leftarrow$ سنستخدم الـ ascii لتخزينها.

الحل:

```
.data
st1: .ascii "ABCDE"      # storing capital letters
st2: .space 5             # reserving space to store the small letters

.text
la $s1, st1               # $s1 = the beginning address of st1
la $s2, st2               # $s2 = the beginning address of st2

Loop:
lb $t0, 0($s1)            # $t0 <-- M[$s1]
beq $t0, $0, End          # if ($t0 == 0) {go to "End"} else {continue sequentially}
addi $t1, $t0, 32         # $t1 = $t0 + 32 (convert to small letter)
sb $t1, 0($s2)            # $t1 --> M[$s2] {storing the small letter}
addi $s1, $s1, 1          # $s1 = $s1 + 1 (go to the next capital letter)
addi $s2, $s2, 1          # $s2 = $s2 + 1 (go to the next empty space in st2)
j Loop                    # repeat the loop

End:
```



Life is like riding a bicycle, to keep your balance you have to keep moving.

- Albert Einstein

محتوى الذاكرة قبل تنفيذ البرنامج

Data Segment					
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)
0x10010000	D C B A	\0 \0 \0 E	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

محتوى الذاكرة بعد تنفيذ البرنامج

Data Segment					
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)
0x10010000	D C B A	b a \0 E	\0 e d c	\0 \0 \0 \0	\0 \0 \0 \0

شرح الكود:

السطر 2: تخزين سلسلة محارف st1 تحوي القيم "ABCDE" باستخدام asciiz والتي ستضيف null byte إلى نهايتها عند التخزين.

السطر 3: قمنا بحجز مساحة 5 bytes لتخزين قيم الأحرف الصغيرة بعد تحويلها.

السطر 5: خزنا عنوان بداية السلسلة st1

السطر 6: خزنا عنوان بداية st2 في \$s2

السطر 9: نخزن محتوى \$s1 في \$t0

السطر 10: إذا كان \$t0 == 0 وصلنا إلى نهاية st1 (نهاية السلسلة) ⇒ ننتقل إلى End وننهي البرنامج.
إذا كان \$t0 != 0 ⇒ نكمل الكود بالتسلسل.

السطر 11: نقوم بجمع محتوى السجل \$t0 مع العدد 32 حتى نحصل على قيمة الحرف الصغير ascii وتخزينه في \$t1

السطر 12: نخزن قيمة \$t1 في عنوان الذاكرة M[\$s2]

السطر 13: نجمع 1 إلى السجل \$s1، لينتقل إلى الحرف التالي في st1.

السطر 14: نجمع 1 إلى السجل \$s2، لينتقل إلى البايت الفارغ في st2.

السطر 15: نكرر Loop



■ ملاحظة

إذا كان التخزين باستخدام `asciiz`. فيمكننا كسر الحلقة عند الوصول إلى القيمة 0. (كما في المثال السابق) أما إذا كان التخزين باستخدام `ascii`. فيجب أن يتم إعطاء عدد عناصر السلسلة في السؤال وعلى أساسه يمكن كسر الحلقة.

■ تمرين 2:

اكتب برنامج بلغة تجميع MIPS يقوم بجمع العدد 32 إلى عناصر مصفوفة أعداد صحيحة مخزنة في الذاكرة بدءاً من العنوان القاعدي `$S1`، و احفظ النواتج بدءاً من موقع الذاكرة ذي العنوان القاعدي في `$S2` علماً أن المصفوفة تنتهي بـ 0.

الحل 1: باعتبار أننا نريد جمع العدد 32 إلى جميع عناصر المصفوفة عدا الصفر. هذا الحل مشابه لحل التمرين السابق لكن مع اختلافات بسيطة حيث أن هنا نتعامل مع كلمات `word`. وليس بايتات `byte`. على عكس التمرين السابق. كل `sb` تصبح `sw`، كل `lb` تصبح `lw`، والتخزين يتم باستخدام `word`. وللانتقال للعنصر التالي نجمع 4 وليس 1.

.data

```
arr1: .word 1, 2, 3, 4, 0    # storing the array which ends with 0
arr2: .space 20             # reserving space for storing the answer
```

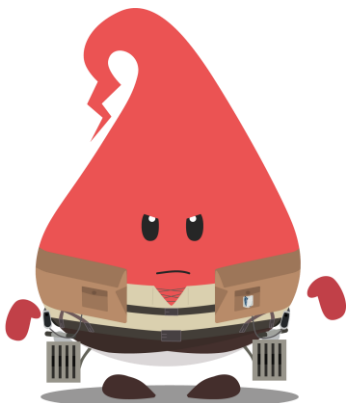
.text

```
la $s1, arr1                # $s1 = the beginning address of arr1
la $s2, arr2                # $s2 = the beginning address of arr2
```

Loop1:

```
lw $t0, 0($s1)              # $t0 <-- M[$s1]
beq $t0, $zero, End         # if ($t0 == 0) {go to "End"} else {continue sequentially}
addi $t1, $t0, 32           # $t1 = $t0 + 32
sw $t1, 0($s2)              # $t1 --> M[$s2] (storing the answer)
addi $s1, $s1, 4            # $s1 = $s1 + 4 (go to the next element)
addi $s2, $s2, 4            # $s2 = $s2 + 4 (go to the next empty byte)
j Loop1                     # repeat Loop1
```

End:



If you win, you live, if you lose, you die, if you don't fight, you can't win

- Eren (Attack on Titan)

محتوى الذاكرة قبل تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	2	3	4	0	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

محتوى الذاكرة بعد تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	2	3	4	0	33	34	35
268501024	36	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

الطريقة 2: باعتبار اننا نريد جمع العدد 32 إلى جميع عناصر المصفوفة

.data

arr1: .word 1, 2, 3, 4, 0

storing the array which ends with 0

arr2: .space 20

reserving space for storing the answers

.text

la \$s1, arr1

\$s1 = the beginning address of arr1

la \$s2, arr2

\$s2 = the beginning address of arr2

Loop2:

lw \$t0, 0(\$s1)

\$t0 <-- M[\$s1]

addi \$t1, \$t0, 32

\$t1 = \$t0 + 32

sw \$t1, 0(\$s2)

\$t1 --> M[\$s2] (storing the answer)

addi \$s1, \$s1, 4

\$s1 = \$s1 + 4 (go to the next element)

addi \$s2, \$s2, 4

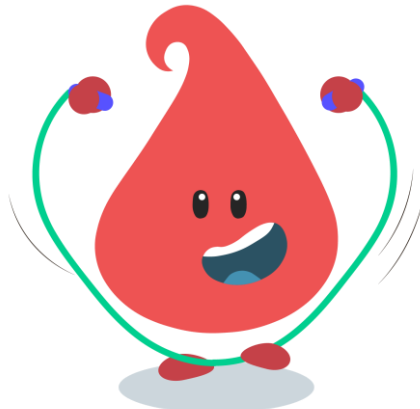
\$s2 = \$s2 + 4 (go to the next empty space)

bne \$t0, \$zero, Loop2

if (\$t0 != 0) {go to Loop2} else {end the code}

محتوى الذاكرة بعد تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	2	3	4	0	33	34	35
268501024	36	32	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0



■ تمرين 3:

ما هو عمل البرنامج التالي وما قيمة \$v0 عند انتهاء التنفيذ:

```
li $t0, 1          # $t0 = 1
li $t1, 0          # $t1 = 0

loop:
add $t1, $t0, $t1  # $t1 = $t0 + $t1
addi $t0, $t0, 1   # $t0 = $t0 + 1
slti $t2, $t0, 7   # if ($t0 < 7) {$t2 = 1} else {$t2 = 0}
bne $t2, $zero, loop # if ($t2 != 0) {go to loop} else {continue sequentially}
move $v0, $t1      # $v0 = $t1
end:
```

الحل:

شرح: نلاحظ أن \$t1 يبدأ من القيمة 1 ويزداد بمقدار 1 مع كل اعادة للحلقة، ويتم جمع قيمة \$t1 إلى \$t0.

تنكسر الحلقة عندما \$t1 < 7 أي عندما \$t1 == 7. وفي نهايتها يتم اسناد قيمة \$t1 إلى \$v0.

عمل البرنامج: جمع الأعداد من 1 إلى 6 بواسطة حلقة.

قيمة \$v0:

$$\$v0 = 1 + 2 + 3 + 4 + 5 + 6 = (21)_{10} = (0x15)_{16}$$

■ تمرين 4:

ما هو عمل البرنامج التالي وما قيمة \$v0 عند انتهاء التنفيذ.

```
addi $s1, $0, 0    # $s1 = 0
addi $s0, $0, 1    # $s0 = 1
addi $t0, $0, 21   # $t0 = 21
loop:
slt $t1, $s0, $t0  # if ($s0 < $t0) {$t1 = 1} else {$t1 = 0}
beq $t1, $0, done  # if ($t1 == 0) {go to "done"} else {continue sequentially}
add $s1, $s1, $s0  # $s1 = $s1 + $s0
add $s0, $s0, $s0  # $s0 = $s0 + $s0
j loop             # repeat the loop
done:
move $v0, $s1
```

الحل:

شرح: نلاحظ:

- قيمة \$t1 = 21 ثابتة طول الكود
- قيمة \$s0 تزداد كل مرة الضعف

- نقوم بجمع قيمة \$s0 إلى \$s1 كل مرة
- شرط كسر الحلقة هو أن تصبح \$s0 ≥ \$t0 أي \$s0 ≥ 21
- ⇐ سنقوم بجمع قيمة \$s0 إلى \$s1 ثم تزيد قيمة \$s0 الضعف ثم نكرر الحلقة، فستكون قيم \$s0 هي:
 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$
 عندما يصبح \$s0 = 32 فستنكسر الحلقة ونذهب إلى done.
- عمل البرنامج: جمع مضاعفات الـ 2 حتى الـ 16 (حيث $16^2 > 21$)
- قيمة \$v0: $v0 = (1 + 2 + 4 + 8 + 16 + 32)_{10} = (0x1F)_{16}$
- الكود بلغة عالية المستوى:

```
int sum = 0;           // sum = $s1
int i;                 // i = $s0
for (i = 1; i < 21; i *= 2)
    sum += i;
```

■ تمرين 5:

ما هو عمل البرنامج التالي وما هي قيمة \$s1 عند انتهاء البرنامج:

```
add $s1, $0, $0        # $s1 = 0
addi $s0, $0, 2        # $s0 = 2
addi $t0, $0, 8        # $t0 = 8
for:
beq $s0, $t0, done     # if ($s0 == $t0) {go to "done"} else {continue sequentially}
add $s1, $s1, $s0      # $s1 = $s1 + $s0
addi $s0, $s0, 1       # $s0 = $s0 + 1
j for                  # repeat the "for"
done:
```

الحل:

شرح: نلاحظ:

- قيمة \$t0 = 8 ثابتة طول الكود
- يتم زيادة \$s0 بمقدار 1 مع كل تكرار لـ for
- شرط كسر الحلقة هو ان تكون \$s0 = \$t0 ⇐ ستكسر عندما \$s0 = 8
- يتم جمع قيم \$s0 إلى \$s1 بعد التحقق من شرط الحلقة
- عمل البرنامج: جمع القيم من 2 إلى 7 وتخزين الناتج في \$s1
- قيمة \$s1: $s1 = 2 + 3 + 4 + 5 + 6 + 7 = (27)_{10} = (1B)_{16}$

■ التمرين 6:

ما هو عمل البرنامج التالي:

.data

arr1: .word 1, 2, 3, 4, 5

arr2: .word 0:5

.text

la \$s1, arr1

la \$s2, arr2

li \$t0, 0 # \$t0 = 0

li \$t2, 4 # \$t2 = 4

loop:

lw \$t1, 0(\$s1) # \$t1 <-- M[\$s1]

sub \$t1, \$0, \$t1 # \$t1 = 0 - \$t1

sw \$t1, 0(\$s2) # \$t1 --> M[\$s2]

addi \$s1, \$s1, 4 # \$s1 = \$s1 + 4

addi \$s2, \$s2, 4 # \$s2 = \$s2 + 4

addi \$t0, \$t0, 1 # \$t0 = \$t0 + 1

bne \$t0, \$t2, loop # if (\$t0 != \$t2) {repeat "loop"} else {end the code}

end:

الحل:شرح: نلاحظ:

- في كل مرة نأخذ المحتوى M[\$s1] ونعكسه ثم نخزنه في M[\$s2]

- بعدها ننتقل إلى العنصر التالي إذ نضيف 4 لكل من \$s1 و \$s2

- نضيف 1 إلى \$t0 كل مرة

- شرط التوقف يتم فحصه بعد هذه التعليمات، وهو أن تكون \$t0 == \$t2

⇐ سيتوقف البرنامج عند \$t0 = 4، أي أننا سنأخذ فقط أول 4 عناصر من المصفوفة ونعكسهم.

عمل الكود: تحميل أول 4 عناصر من المصفوفة arr1 وإيجاد معكوسهم ثم تخزينهم في المصفوفة arr2

محتوى الذاكرة بعد تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	1	2	3	4	5	-1	-2	-3
0x10010020	-4	0	0	0	0	0	0	0

تمرين 7:

اكتب برنامج (باستخدام الحلقات)، يقوم بتحميل عناصر مصفوفى أعداد صحيحة مخزنة في الذاكرة بدءاً من العنوان القاعدي \$s1، وآخر عنصر فيها هو العدد 222، فإن كان العنصر موجباً توجد ضعفيه، وإن كان سالباً نوجد 3 أضعافه، ويتم تخزين القيم الناتجة إلى مواقع الذاكرة بدءاً من العنوان القاعدي \$s2 التالي للعنوان 222.

محتوى الذاكرة قبل تنفيذ البرنامج

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	-2	3	4	-5	222	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

محتوى الذاكرة بعد تنفيذ البرنامج

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	-2	3	4	-5	222	2	-6
268501024	6	8	-15	444	0	0	0	0
268501056	0	0	0	0	0	0	0	0

حل اربسييزي:

طريقة 1: ننشئ عداد counter حيث يكون شرط التوقف هو تكرار الحلقة عدد مرات معين.

.data

arr1: .word 1,-2,3,4,-5, 222

arr2: .word 0:6

.text

la \$s1, arr1

la \$s2, arr2

li \$t0, 222 *# \$t0 = 222 (the last element)*

loop:

lw \$t2, 0(\$s1) *# \$t2 <- M[\$s1] (load the element)*

blt \$t2, \$0, negative *# if \$t2 < 0 --> go to negative, else --> continue to positive*

positive:

add \$t3, \$t2, \$t2 *# \$t3 = \$t2 * 2*

j after

negative:

```
add $t3, $t2, $t2
add $t3, $t3, $t2      # $t3 = $t2 * 3
```

after:

```
sw $t3, 0($s2)          # $t3 -> M[$s2] (store the result)
addi $s1, $s1, 4         # $s1 = $s1 + 4 (jump to the next element in arr1)
addi $s2, $s2, 4         # $s2 = $s2 + 4 (jump to the next element in arr2)
bne $t1, $t0, loop       # if $t0 != $t1 --> repeat, else --> end the code
# The code will end after reaching 222
```

end:

طريقة 2: يكون شرط التوقف هو الوصول إلى العنصر 222.

.data

```
arr1: .word 1,-2,3,4,-5, 222
arr2: .word 0:6
```

.text

```
la $s1, arr1
la $s2, arr2
```

```
li $t0, 222              # $t0 = 222 (the last element)
```

loop:

```
lw $t2, 0($s1)           # $t2 <- M[$s1] (load the element)
blt $t2, $0, negative     # if ($t2 < 0) {go to "negative"} else {continue to positive}
```

positive:

```
add $t3, $t2, $t2        # $t3 = $t2 * 2
j after
```

negative:

```
add $t3, $t2, $t2
add $t3, $t3, $t2        # $t3 = $t2 * 3
```

after :

```
sw $t3, 0($s2)          # $t3 -> M[$s2] (store the result)
addi $s1, $s1, 4         # $s1 = $s1 + 4 (jump to the next element in arr1)
addi $s2, $s2, 4         # $s2 = $s2 + 4 (jump to the next element in arr2)
```

```

bne $t2, $t0, loop      # if ($t0 != $t1) {repeat "loop"} else {end the code}
# The code will end after reaching 222
end:

```

■ تمرين 8:

- اكتب برنامجًا بلغة التجميع MIPS يقوم بالتصريح عن مصفوفة من 5 أعداد صحيحة ثم:
1. يوجد ضعفي أول عنصر منها ويخزنه في آخر موقع فيها،
 2. يقارن بين ثاني وثالث عنصر منها، فإذا كان:
 - متساويين: يخزن 1 مكان العنصر الرابع
 - إن كان الثاني أكبر تمامًا من الثالث: يخزن 2 مكان العنصر الرابع
 - وإلا يخزن العدد 3 مكان العنصر الرابع
- غير قيم عناصر المصفوفة بحيث تحقق جميع الحالات السابقة.
- يمكن استخدام تعليمات زائفة في البرنامج مع ضرورة وضع مكافئها الحقيقي في التعليق.

حل أربسييزي:

```

.data
arr: .word -1, 2, 2, 4, -5

.text
la $s1, arr                # real : lui $at, upper(arr)   ori $at, $s1, lower(arr)

# arr[4] = arr[0] * 2
lw $t0, 0($s1)
add $t0, $t0, $t0
sw $t0, 16($s1)            # offset = 4 jumps * 4 = 16

# if (arr[1] == arr[2]) {arr[4] = 1}
lw $t1, 4($s1)
lw $t2, 8($s1)

bne $t1, $t2, not_equal
li $s2, 1                  # real : addiu $s2, $0, 1
sw $s2, 12($s1)
j end

not_equal:
bgt $t1, $t2, greater     # real : slt $at, $t2, $t1   bne $at, $0, greater
# else {arr[4] = 3}
li $s2, 3                  # real : addiu $s2, $0, 3

```

```
sw $s2, 12($s1)
```

```
j end
```

```
greater:
```

```
# if arr[1] > arr[2] --> arr[4] = 2
```

```
li $s2, 2 # real : addiu $s2, $0, 2
```

```
sw $s2, 12($s1)
```

```
end:
```

محتوى الذاكرة قبل تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	2	4	-5	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

محتوى الذاكرة بعد تنفيذ البرنامج:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	2	1	-2	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

نلاحظ ان القيم السابقة تحقق الحالة الأولى (حالة التساوي)

فإذا قمنا بتغيير قيم المصفوفة إلى التالي:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	-2	4	-5	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

فتكون الذاكرة بعد تنفيذ الكود (تحقق الحالة الثانية):

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	-2	2	-2	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

ولنتحقق الحالة الثالثة نجعل القيم كالتالي:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	3	4	-5	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0

فتكون الذاكرة بعد تنفيذ الكود:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	-1	2	3	3	-2	0	0	0
268501024	0	0	0	0	0	0	0	0
268501056	0	0	0	0	0	0	0	0